

Competitive Programming

Section 46: Rerooting and Advanced Tree DP

Competitive Programming Course

2026-07-10

Table of contents

1	Rerooting viewpoint	3
1.1	Sum of distances	3
1.2	Farthest distance from every node	3
1.3	Paths through a node	4
1.4	Rerooting checklist	4
1.5	Worked sum-distance transition	4
1.6	General rerooting pattern	5
1.7	Tree shapes to test	5
2	Exercises	6
3	Rerooting proof habit	7

1 Rerooting viewpoint

Rerooting asks for an answer for every possible root. First compute information inside each child subtree. Then move the root across an edge and update the answer by subtracting the child's contribution and adding the outside contribution. Diameter endpoints are another rerooting shortcut for farthest distance queries.

1.1 Sum of distances

Root the tree at 1. During the first DFS compute:

```
sub[v]  = size of v's subtree
down[v] = sum of distances from v to nodes in v's subtree
```

For a child to of v :

```
sub[v] += sub[to]
down[v] += down[to] + sub[to]
```

The $+ sub[to]$ appears because every node in the child subtree is one edge farther from v than from to .

Now suppose we know $ans[v]$, the sum of distances from v to all nodes. Move the root across edge v - to :

```
ans[to] = ans[v] - sub[to] + (n - sub[to])
```

Nodes inside to 's subtree become one step closer; there are $sub[to]$ of them. All other nodes become one step farther; there are $n - sub[to]$.

1.2 Farthest distance from every node

A useful shortcut: in an unweighted tree, the farthest node from any vertex is one of the two endpoints of a diameter. Find a diameter endpoint a by BFS/DFS from any node, then find the opposite endpoint b by BFS/DFS from a .

For every vertex v :

```
answer[v] = max(dist(a, v), dist(b, v))
```

This avoids a heavier rerooting DP and is a standard contest trick.

1.3 Paths through a node

Remove a node v from the tree. The remaining connected components have sizes:

- each child subtree size;
- the “parent side” size $n - \text{sub}[v]$.

A path between two distinct nodes passes through v exactly when the two endpoints lie in different components after removing v , or one endpoint is v itself.

Count unordered pairs from different components:

```
pairs = total_choose_2 - sum(component_size_choose_2)
```

If the statement counts paths with v as one endpoint too, add $n-1$; in this course exercise the local checker defines the exact requested convention in the problem README.

1.4 Rerooting checklist

- State what each subtree value means before coding.
- Derive the edge transition with a small drawing.
- Watch whether the answer counts nodes, edges, ordered pairs, or unordered pairs.
- Use iterative DFS if recursion depth is risky in Python.

1.5 Worked sum-distance transition

Tree:

```
1 - 2
 |
3
```

Root at 1.

```
sub[2]=1, sub[3]=1, sub[1]=3
ans[1]=dist(1,2)+dist(1,3)=2
```

Move root from 1 to child 2:

```
ans[2] = ans[1] - sub[2] + (n-sub[2])
        = 2 - 1 + 2
        = 3
```

Indeed:

```
dist(2,1)+dist(2,3)=1+2=3
```

The formula is not a trick; it counts which nodes got one step closer and which got one step farther.

1.6 General rerooting pattern

Many rerooting problems have two passes:

```
dfs_down(v): compute information from children into v
dfs_up(v): pass information from parent/outside into children
```

For each edge v -child, ask:

```
What does child lose when it stops seeing v as parent?
What does child gain from the outside of its subtree?
```

If the merge operation is not easily invertible, use prefix/suffix merges over children so each child can receive “all siblings except itself” in $O(1)$ after preprocessing.

1.7 Tree shapes to test

Rerooting formulas should be tested on:

```
single node
chain
star
balanced binary tree
```

Chains expose off-by-one distance bugs. Stars expose whether the parent-side component was counted. Single-node trees expose assumptions that every node has a parent or child.

2 Exercises

3 Rerooting proof habit

Root once at 1. First compute values flowing from children to parent; then pass the complement value from parent to child. For sum distances, `down[v]` is the sum to its subtree and `size[v]` its size. Moving the root across edge `v-u` changes the answer by `n-2*size[u]`: `size[u]` nodes become one closer and all others one farther. Test a chain, a star, and `n=1`.

- A. Sum Distances: For every node, print the sum of distances to all other nodes.
- B. Farthest Node Distance: For every node, print its distance to the farthest node.
- C. Paths Through Node: For every node, count unordered pairs of distinct nodes whose path passes through it.